```
#NAME : ADITYA BHULANGE
#ROLL NO. : SA273
```

```
#Project Title:Climate Change Data Analysis and Prediction
```

```
#Problem Statement:
#I am analyzing a climate change dataset to understand how different environmental factors like temperature, CO₂ levels, and pollution
```

```
#Dataset Description

#Dataset: Beginner Climate Change Dataset
#Source: (mention where you got it – Kaggle/local file)
#Rows: 1200
#Columns: 20

#The dataset includes features like temperature, CO₂ emissions, humidity, pollution levels, and other environmental indicators.
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files Beginner_C…_Rows.csv

Beginner_Climate_Change_Dataset_20_Features_1200_Rows.csv(text/csv) - 126936 bytes, last modified: 4/2/2026 - 100% done

```
Saving Beginner_Climate_Change_Dataset_20_Features_1200_Rows.csv to Beginner_Climate_Change_Dataset_20_Features_1200_Rows (2).csv
```

```
df = pd.read_csv('Beginner_Climate_Change_Dataset_20_Features_1200_Rows.csv')
```

```
df.head()
```

	year	country	global_avg_temperature	temperature_anomaly	max_temperature	min_temperature	co2_concentration_ppm	annual_rainfall
0	2018	Germany	14.03	1.16	37.02	4.31	387.85	81
1	2008	India	15.03	1.05	32.25	-0.44	407.24	73
2	1994	Pakistan	14.86	1.24	41.57	1.08	450.54	198
3	2022	USA	15.29	1.19	32.17	1.68	415.42	116
4	1987	Australia	13.75	1.32	40.99	10.38	403.42	117

Next steps: Generate code with df New interactive sheet

```
df.shape
df.dtypes
```

	0
year	int64
country	object
global_avg_temperature	float64
temperature_anomaly	float64
max_temperature	float64
min_temperature	float64
co2_concentration_ppm	float64
annual_rainfall_mm	float64
sea_level_rise_mm	float64
sea_surface_temperature	float64
heatwave_days	int64
drought_index	float64
flood_events_count	int64
forest_cover_percent	float64
deforestation_rate	float64
fossil_fuel_consumption	float64
renewable_energy_share	float64
air_quality_index	int64
predicted_temperature_2050	float64
climate_risk_index	float64

dtype: object

```
df.isnull().sum()
```

	0
year	0
country	0
global_avg_temperature	0
temperature_anomaly	0
max_temperature	0
min_temperature	0
co2_concentration_ppm	0
annual_rainfall_mm	0
sea_level_rise_mm	0
sea_surface_temperature	0
heatwave_days	0
drought_index	0
flood_events_count	0
forest_cover_percent	0
deforestation_rate	0
fossil_fuel_consumption	0
renewable_energy_share	0
air_quality_index	0
predicted_temperature_2050	0
climate_risk_index	0

dtype: int64

```
df.fillna(df.mean(numeric_only=True), inplace=True)
#Missing values were replaced with the mean because it preserves the dataset size and avoids bias.
```

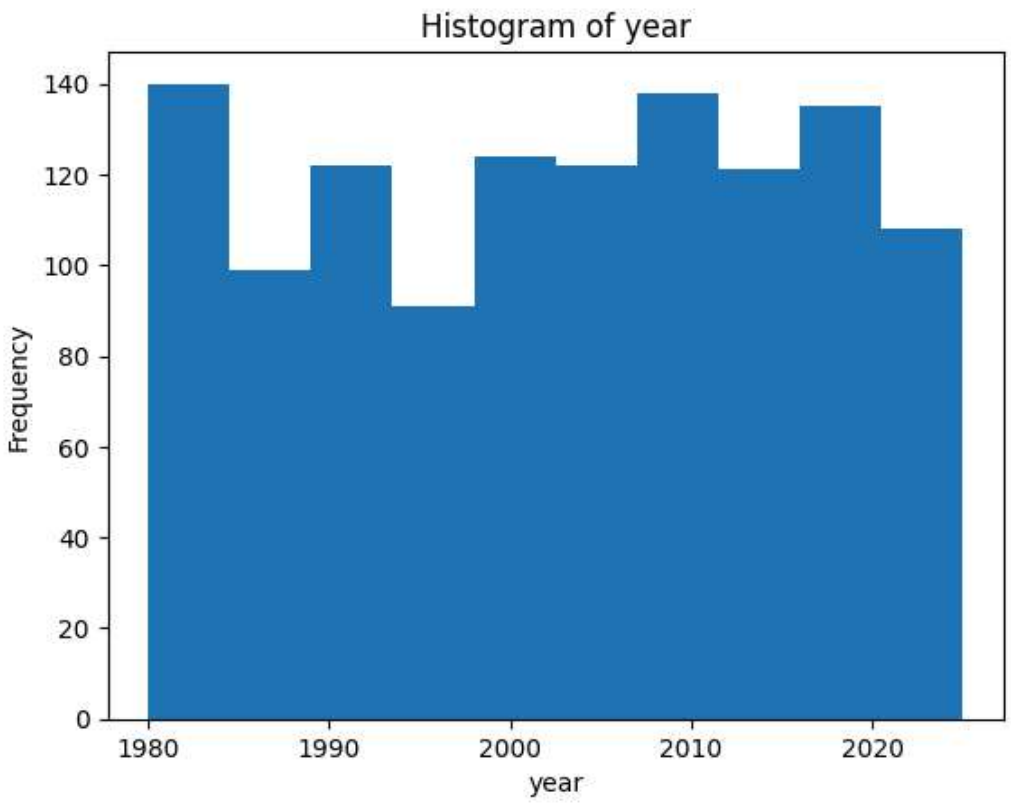
```
df.drop_duplicates(inplace=True)
```

```
col1 = df.select_dtypes(include=np.number).columns[0]
col2 = df.select_dtypes(include=np.number).columns[1]
```

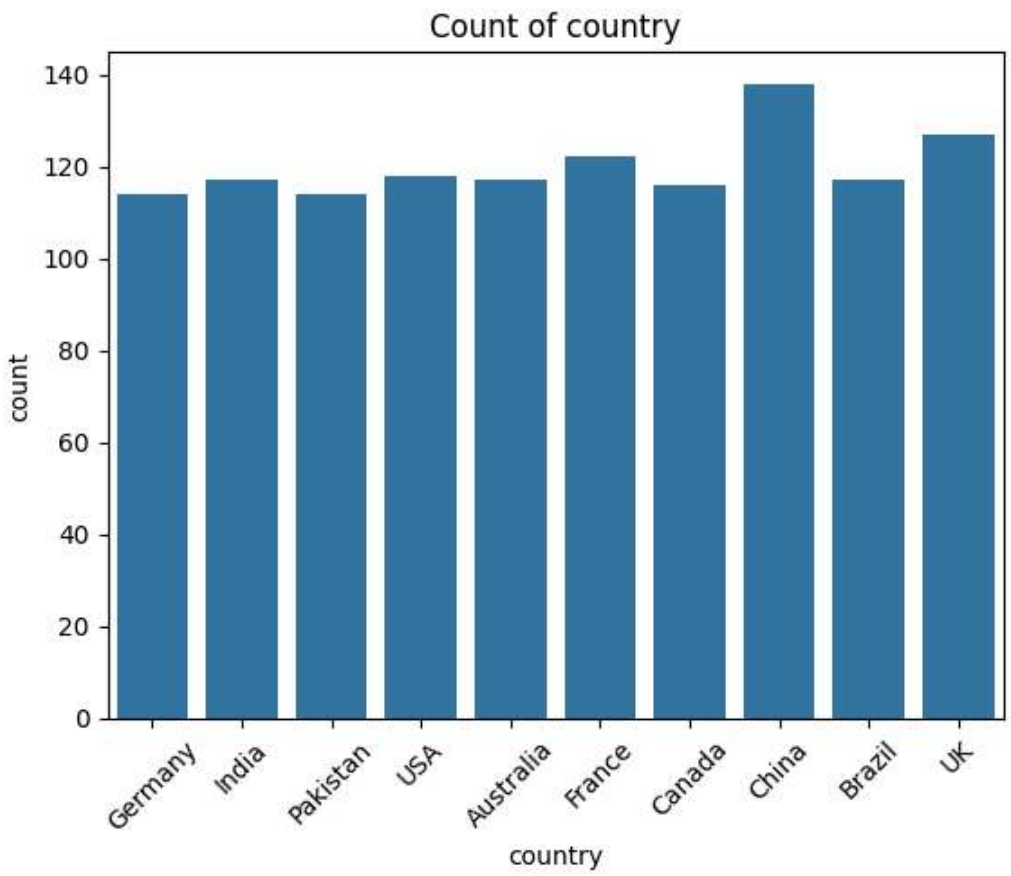
```
def stats(col):
    return {
        "Mean": df[col].mean(),
        "Median": df[col].median(),
        "Mode": df[col].mode()[0],
        "Std": df[col].std(),
        "Variance": df[col].var(),
        "Range": df[col].max() - df[col].min(),
        "Mid-range": (df[col].max() + df[col].min()) / 2
    }

print(col1, stats(col1))
print(col2, stats(col2))
#The dataset had some missing values which were filled using mean values. Duplicate rows were removed. The dataset is now clean and re
```

```
year {'Mean': np.float64(2002.4525), 'Median': 2003.0, 'Mode': np.int64(2005), 'Std': 13.101492968933428, 'Variance': 171.6491180150120
global_avg_temperature {'Mean': np.float64(14.492849999999999), 'Median': 14.51, 'Mode': np.float64(14.52), 'Std': 0.5945880843000981,
```

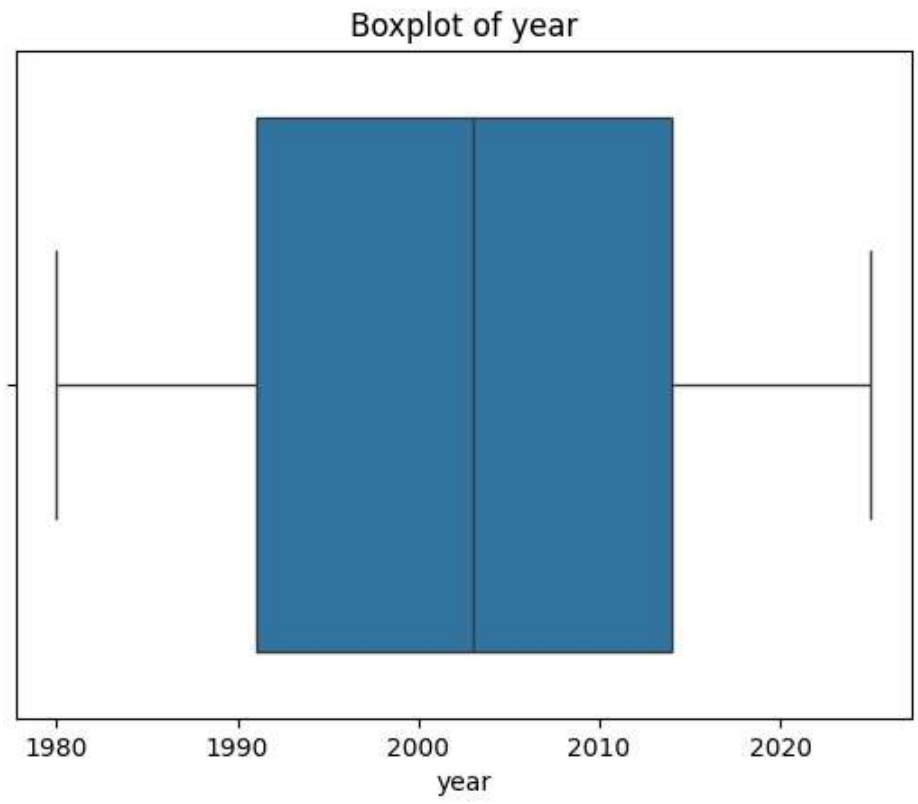
```
plt.hist(df[col1])
plt.title("Histogram of " + col1)
plt.xlabel(col1)
plt.ylabel("Frequency")
plt.show()

#The histogram shows how values of this feature are distributed.
```



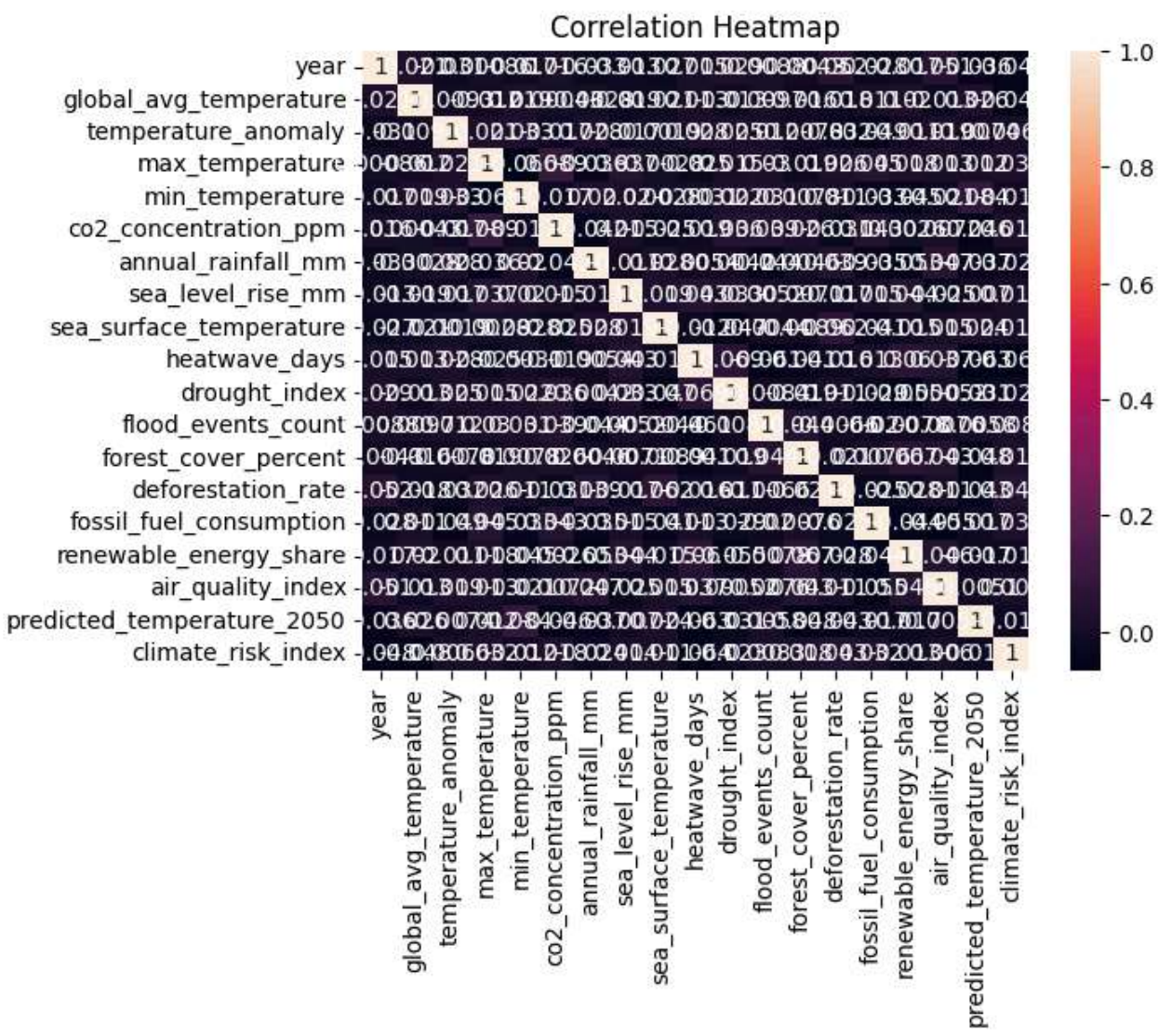

```
cat_col = df.select_dtypes(include='object').columns[0]

sns.countplot(x=df[cat_col])
plt.title("Count of " + cat_col)
plt.xticks(rotation=45)
plt.show()

#This chart shows how frequently each category appears.
```



```
sns.boxplot(x=df[col1])
plt.title("Boxplot of " + col1)
plt.show()

#The boxplot shows spread and possible outliers.
```




```
sns.heatmap(df.corr(numeric_only=True), annot=True)
plt.title("Correlation Heatmap")
plt.show()

#The heatmap shows relationships between numeric features.
```




```
le = LabelEncoder()

for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])
```

```
# Import ML libraries
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Encoding categorical columns
le = LabelEncoder()
for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])
```

```
# Select target (first numeric column)
target = df.select_dtypes(include='number').columns[0]

# Split features and target
X = df.drop(columns=[target])
y = df[target]

# Train-test split (80:20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluation
mse = mean_squared_error(y_test, y_pred)

print("Target Column:", target)
print("Mean Squared Error:", mse)

#The model predicts the selected variable based on other features. Lower error means better prediction accuracy.
```

```
Target Column: year
Mean Squared Error: 157.45110910636546
```

```
#Insight:
#Example:
#The heatmap shows a strong correlation between temperature and CO₂ levels.
#The histogram indicates that most temperature values are concentrated in a specific range.
#The boxplot reveals some extreme outliers in pollution levels.
```